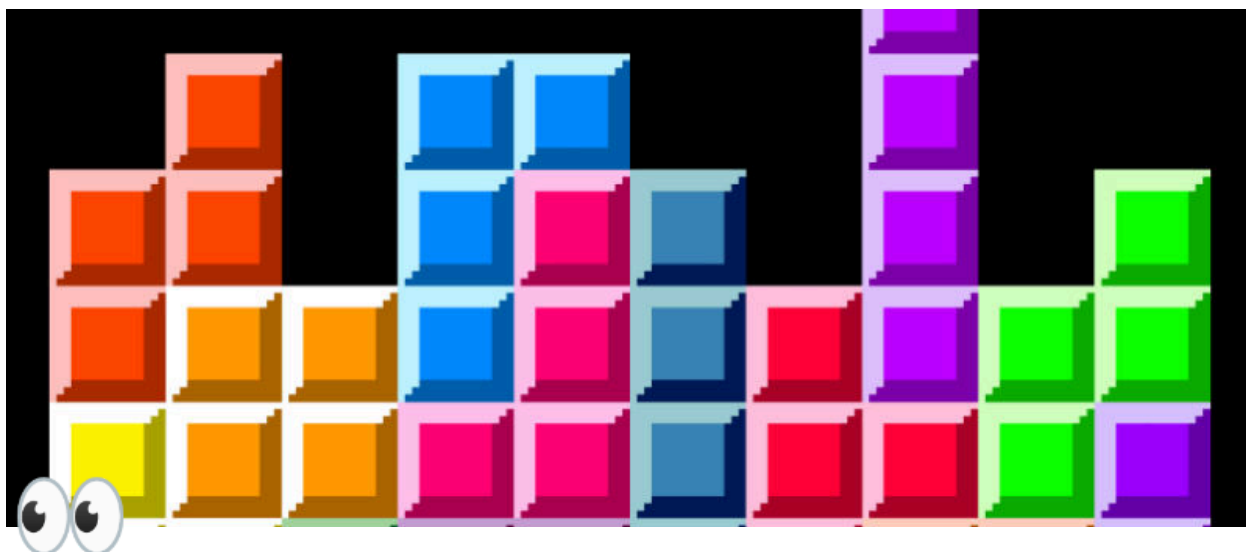




HW2 - 雙人即時對戰遊戲設計與實作（雙人俄羅斯方塊）

作業目標

以原生 Socket 為基礎，完成以下三項核心需求（集中式遊戲伺服器、雙人對戰、獨立 DB 服務）：

Central Lobby Server（集中式大廳 | CLI）

- 註冊 / 登入 / 登出
- 建立/加入房間（每房 2 人）
- 在線列表與房間列表
- 邀請/接受邀請加入房間
- 開始對戰：Lobby 負責建立 game server，自動挑選可行 port

雙人制即時俄羅斯方塊（Game Server 在課程機上，Client GUI 在本地）

- 架構：Client-Server。

- **Game Server (CLI)**：跑在課程機，維護雙方遊戲狀態 (Server authority)，判斷遊戲邏輯。
- **Game Client (GUI)**：跑在學生本地，只傳操作、顯示畫面，不判斷邏輯。
- 玩法：各自落子、各自計分，**基礎版無攻擊 (垃圾行)**。
- 顯示：**必須顯示對手棋盤 (縮小版即可)**。
- 觀戰 (選做)：只讀模式，從 Game Server 收快照即可。

簡易 Database Server (獨立行程)

- **TCP + JSON** 自製輕量 NoSQL 服務，支援 `User`、`GameLog` CRUD/查詢。
- 所有資料操作皆透過 **DB Server 的 Socket API**，不得直寫檔案繞過它。
- 可用 CSV/SQLite/txt/JSON 等作為底層存儲，但對使用方一律走 Socket API。

前置要求

TCP連線請自訂以下通訊協定: Length-Prefixed Framing Protocol, UDP可自由設計

什麼是 Length-Prefixed Framing Protocol ?

長度前綴分框協議是一種 TCP 通訊格式，每個訊息的開頭都會附帶一個固定大小的**長度欄位** (header)，用來表示接下來訊息本體的位元組數。

例子：

要傳送兩個訊息 `"Hello"` 與 `"World"`，發送順序如下：

1. 先傳送一個 4 位元組的長度欄位，表示 `"Hello"` 長度為 5 位元組。
2. 傳送 `"Hello"` (5 位元組)。
3. 傳送一個 4 位元組的長度欄位，表示 `"World"` 長度為 5 位元組。
4. 傳送 `"World"` (5 位元組)。

為什麼要用這種協議？

TCP 是**以串流為基礎**的傳輸協議，它將資料當作連續的位元組流來處理，不保留訊息邊界，這會造成：

- **黏包 (packet sticking)**：多次 `send()` 可能被合併，導致接收端一次 `recv()` 收到多個訊息（例如 `"HelloWorld"`）。
- **拆包 (packet fragmentation)**：一次 `send()` 的資料可能被拆成多次 `recv()` 才能收完整。

加上長度欄位後，接收端就能準確分割並重組每個完整訊息。

例子：

傳送 `"Hello"`（5 位元組）時：

- **長度欄位**：`00000005`（4 位元組 `uint32`，網路位元序，大端，數值為 5）。
- **訊息本體**：`"Hello"`（UTF-8 編碼，5 位元組）。
- **傳輸格式**：`[00000005][Hello]`

實作要求

框架格式：

```
[4-byte length (uint32, network byte order)] [body: length bytes (custom format)]
```

- **長度欄位**：4 位元組無號整數（`uint32`），**網路位元序**（大端）。
 - C 用 `htonl()`
 - Java 用 `ByteBuffer.putInt()`
 - Python 用 `struct.pack('!I', length)`
- **訊息本體**：由長度欄位指定的位元組數，格式自訂（如 UTF-8、JSON 等）。

長度限制：

- 拒絕任何 `length > 64 KiB`（65536 位元組）的訊息，以防止資源濫用。
- 接收端應驗證長度欄位，若超過限制則回報錯誤或關閉連線。

部分 I/O 處理 (Partial I/O Handling)

TCP 的 `send()` / `recv()` 不保證一次就能傳送或接收完整訊息，必須處理部分讀寫：

讀取 (接收端)：

1. 先讀滿 4 位元組長度欄位。
2. 解析為 `uint32` 長度，驗證長度 ($0 < \text{length} \leq 64 \text{ KiB}$)。
3. 再讀滿 `length` 位元組訊息本體。
4. 若資料不完整，繼續等待更多資料。

寫入 (發送端)：

1. 將訊息長度轉成 4 位元組 `uint32` (網路位元序)。
2. 傳送長度欄位。
3. 傳送訊息本體。
4. 若 `send()` 只送出部分資料，必須持續送完剩下的部分。

錯誤處理：

- 若標頭不足 4 位元組，需等待更多資料。
- 若長度欄位無效 (≤ 0 或 $> 64 \text{ KiB}$)，回報錯誤或關閉連線。
- 若 socket 關閉或出錯，需終止。

一、Mock Database Server

傳輸協定與格式

- **Protocol**：TCP socket
- **編碼**：UTF-8
- **資料格式**：每筆請求/回應皆透過 **Length-Prefixed Framing Protocol** 傳輸，內容為一個完整的 JSON 字串

- **10/29更新**：為了同學實作方便，TCP 傳輸的內容不限於 JSON，同學可以用 raw string、自訂格式等方式進行TCP傳輸，並能正確解析即可。而local端的DB儲存格式可以txt、json、csv、sqlite 等方式存檔，同學可依自己實作習慣選擇，只要是透過 DB Server 的 API 查詢更新即可。

基本請求結構

```
{  
  "collection": "User | Room | GameLog",  
  "action": "create | read | update | delete | query",  
  "data": { ... }    // 具體欄位見下  
}
```

參考資料模型

實際 schema 可依需求自行設計，但請在 Report 明確列出並與伺服器對應一致。

- **User** : { id, name, email, passwordHash, createdAt, lastLoginAt }
- **Room** : { id, name, hostUserId, visibility("public"|"private"), inviteList[], status("idle"|"playing"), createdAt }
- **GameLog** (對局摘要與結果) :
{ id, matchId, roomId, users:[userId], startAt, endAt, results:[{userId, score, lines, maxCombo}] }

二、Central Lobby Server

功能需求：

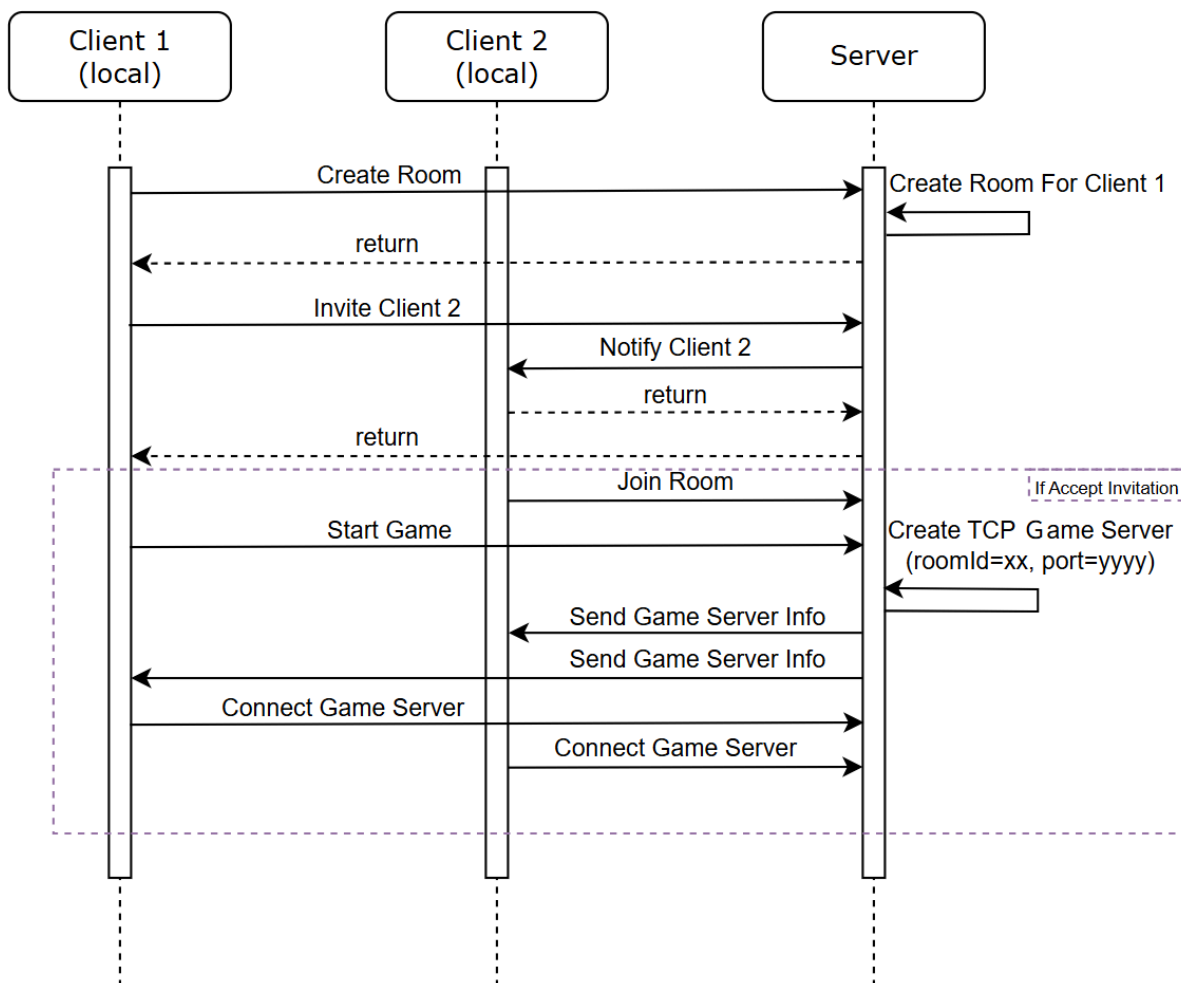
- 使用者註冊 / 登入 / 登出（只能透過DB Server查詢）
- 顯示線上使用者列表（只能透過DB Server查詢）
- 顯示公開房間列表（含是否開放、房主）（只能透過DB Server查詢）
- 建立公開/私有房間；公私有房皆可邀請指定使用者，公開房間所有人皆能加入
- 邀請/接受邀請加入房間(邀請不可被io block住，應該有一個invitation list讓使用者去回復)

- 加入/離開房間
- 開始對戰（只負責配對與分發連線資訊，對戰資料流不經大廳）
- 對戰後必須能回到房間，Game Server 回報 Lobby → Lobby 更新房間、對局DB狀態 → 玩家回到房間，可以再開一局或結束。
- 觀戰入口：觀戰者可選擇房間進入只讀模式，觀戰者只能訂閱 **SNAPSHOT**，不可送 **INPUT** (選做)

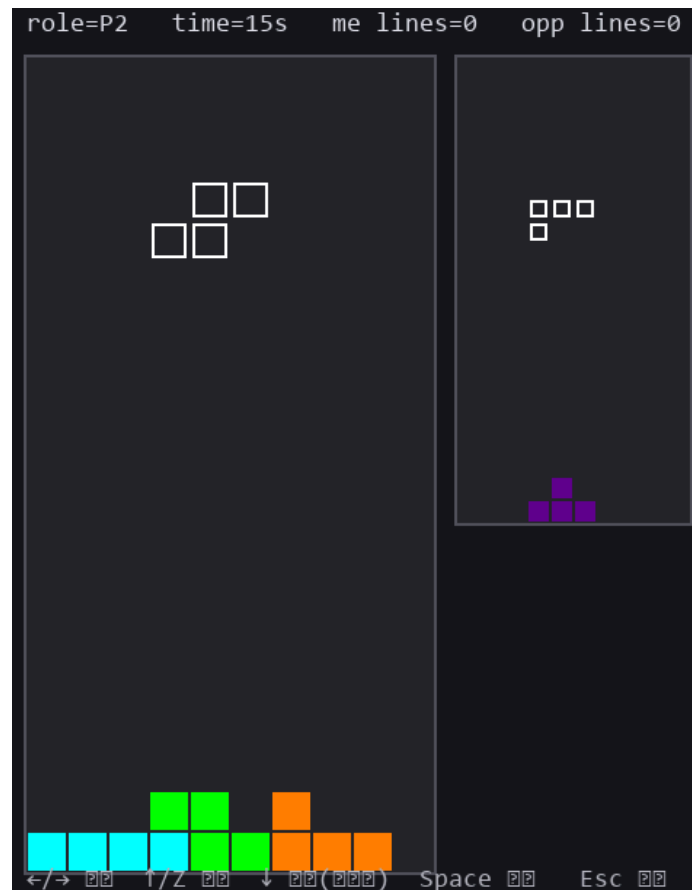
資料流界線

- 配對前：全走 Lobby (TCP + JSON)
- 對戰中：開啟一個額外game server, Lobby 不參與

遊戲流程示意圖



三、即時多人俄羅斯方塊（無須攻擊）



玩法規格（最小可行）

- 人數：2
- 每位玩家各自的棋盤、各自計分與消行，**不互相影響**
- 必須可以看到對手棋盤快照
- 棋盤大小：10 × 20
- 遊戲邏輯必須由Game Server處理再廣播給玩家，不可Client各自處理，Client端只負責渲染
- 支援操作：Left/Right、Rotate、Soft Drop、Hard Drop（選做）、Hold（選做）
- 能力參數：**重力（gravity / level）** 可固定或階段提升
- **無攻擊**：消行不會影響他人

- 結束條件：
 - 計時賽（至少30秒up）（到時比消塊數量）或
 - 存活賽（最後未頂滿者勝）或
 - 固定行數先達標者勝（自由擇一，在 Report 說明）

同步與一致性

1. 種子一致性

- 遊戲開始時由Game Server廣播 `seed` 與 `bagRule`（建議 7-bag + Fisher-Yates），所有玩家用同一亂數序列生成方塊，確保方塊順序一致。

2. 節奏參數廣播(選做)

- Game Server廣播 `gravityPlan`（例如：固定 `dropInterval` 或每 60 秒加快一次），Client端使用單調時鐘驅動落子，僅在收到更新時調整節奏。

3. 延遲抑制

- 傳輸以玩家輸入事件為主，搭配定期狀態快照（例如每 1-2 秒一次）修正分歧。
- 客戶端需有固定渲染緩衝（建議 100-200 ms）平滑顯示他人狀態，減少網路延遲造成的畫面抖動。

4. 觀戰者(選做)

- 觀戰模式僅接收狀態快照，不需處理輸入事件，也需經過延遲抑制以確保畫面穩定。

封包與協定範例

連線管理（玩家 → Game Server）

```
{ "type": "HELLO", "version": 1, "roomId": 123, "userId": 17, "roomToken": "abc123" }
```

連線回應（Game Server → 玩家）

```
{  
  "type": "WELCOME",  
  "role": "P1",  
}
```



```
"seed":987654321,
"bagRule":"7bag",
"gravityPlan":{"mode":"fixed","dropMs":500}
}
```

玩家輸入事件 (Client → Game Server)

```
{ "type":"INPUT", "userId":17, "seq":102, "ts":1234567890, "action":"CW" }
```

狀態快照 (Game Server → Clients)

```
{
  "type":"SNAPSHOT",
  "tick":2201,
  "userId":17,
  "boardRLE":"...壓縮字串...",
  "active":{"shape":"T","x":5,"y":17,"rot":1},
  "hold":"I",
  "next":["O","L","J"],
  "score":12400,
  "lines":9,
  "level":4,
  "at":1234567999
}
```

節奏更新 (Game Server → Clients)

```
{ "type":"TEMPO", "dropMs":420, "effectiveAt":1234568000 }
```

以上欄位皆為示例，可依照實作自行調整設計，但請固定格式並寫入 Report。

四、交付項目

1. **原始碼** 與依賴清單 (`requirements.txt` 或等效)

2. **Report** (清楚說明)

- 系統架構 (Lobby、DB)
- 協定格式 (JSON 欄位、事件流程)
- 同步策略 (輸入事件、快照頻率)
- 玩法 (規則、結束條件、計分方式)

五、評分指標 (滿分120)

若該項相關問題回答不出來，則斟酌扣分

指標	說明	配分
Length-Prefixed Framing Protocol	自訂義TCP通訊協議符合規範	10
DB 設計與正確性	可查詢、結果一致	20
Lobby Server	Lobby 之連線、廣播、房間創建、狀態等	20
遊戲邏輯正確性	能依照最小要求實現遊戲	40
延遲抑制	延遲不影響遊戲體驗(可正常demo即可，小延遲可接受)	5
例外處理	demo流程不因意外操作而卡死	5
UI、創意	UI/特效、遊戲基礎上變化等	10
觀戰	可以看到當前對局	10

六、注意事項

- 不限制任何實作語言。
- 所有連線請走TCP。
- TCP內容不限於使用JSON，可依實作習慣選擇。
- **只可使用標準 Socket API** (TCP)。不可使用 WebSocket / socket.io。
- **不得使用商用或雲端資料庫**；資料須儲存在本機 (CSV/SQLite 皆可，但一律走 DB Server API)。

- 所有使用之第三方 GUI 或輔助函式庫需列在 Report。
- 所有server端應該在系計中server上執行，而client端在local端執行。
- 所有使用到Port請run在10000以上。
- 除了遊戲以GUI呈現以外，Server的溝通只能在終端執行。
- 禁止抄襲，違者雙方該作業以0分計算。
- 任何作業二相關問題歡迎在每周一課程提出或來信（洪崇維：k0310507@gmail.com）